

OOP in JavaScript

Object-Oriented Programming

Complete Reference Notes

Topics Covered

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism
5. Classes (ES6)
6. Objects
7. Prototype & Prototype Chain
8. this Keyword
9. Static Methods
10. Constructor Function
11. Functional Inheritance
12. Method Overriding
13. Method Overloading
14. Object Creation Methods

What is OOP?

Objects • Classes • Principles

OOP is a way of organizing code using:

- Objects → represent real-world entities
- Classes / Functions → blueprints
- Principles → rules that make code reusable & maintainable

Concept	One-Line Memory Trick
Encapsulation	Hide data
Abstraction	Hide logic
Inheritance	Reuse code
Polymorphism	Same method, different behavior
Prototype	Core of JS inheritance

1. Encapsulation

Data Hiding — Protect Internal State

- Combines data + methods into one unit (class/object)
- Prevents direct access to internal data
- Achieved using: #private fields & Closures
- Improves: Security & Maintainability

Important Notes:

- Private variables cannot be accessed outside the class
- Use getter/setter methods to control access
- Helps avoid accidental data modification

Example keywords:

- #variable
- get(), set()

Example:

```
class BankAccount {  
  #balance = 0; // private field  
  
  deposit(amount) {  
    this.#balance += amount;  
  }  
  
  getBalance() {  
    return this.#balance;  
  }  
}  
  
const acc = new BankAccount();  
acc.deposit(1000);  
console.log(acc.getBalance()); // 1000  
  
// console.log(acc.#balance); ❌ Error (private)
```

Output:

1000

2. Abstraction

Hide Complexity — Show Only What Matters

- Shows only essential features
- Hides internal implementation
- User interacts with a simple interface

Important Notes:

- Achieved using: Classes, Functions
- Improves: Code readability & Usability

Real Example:

- Car → You use steering & accelerator
- You don't see engine logic

Example:

```
class Car {  
  start() {  
    this.#igniteEngine();  
    console.log("Car started");  
  }  
  
  #igniteEngine() {  
    console.log("Engine running internally...");  
  }  
}  
  
const c = new Car();  
c.start();
```

Output:

```
Engine running internally...  
Car started
```

3. Inheritance

Reusability — Parent-Child Relationships

- One class inherits properties/methods from another
- Avoids code duplication
- Creates parent-child relationship

Important Notes:

- Use extends keyword
- Use super() to call parent constructor
- Supports method overriding

Types in JS:

- Single inheritance ✓
- Multi-level inheritance ✓
- Multiple inheritance ✗(not directly supported)

Example:

```
class Animal {  
  speak() { return "Animal makes sound"; }  
}
```

```
class Dog extends Animal {  
  bark() { return "Dog barks"; }  
}
```

```
const d = new Dog();  
console.log(d.speak()); // inherited  
console.log(d.bark());
```

Output:

```
Animal makes sound  
Dog barks
```

4. Polymorphism

Many Forms — Same Method, Different Behavior

Polymorphism means "one thing, many forms" — the same method or function behaves differently depending on the context.

Important Notes:

- JavaScript does NOT support traditional method overloading
- Runtime decides which method to call

a. Method Overriding (Runtime Polymorphism)

- A child class provides its own implementation of a method already defined in the parent class
- Same method name, but different behavior

Example:

```
class Animal {  
    speak() { console.log("Animal makes a sound"); }  
}  
  
class Dog extends Animal {  
    speak() { console.log("Dog barks"); }  
}  
  
class Cat extends Animal {  
    speak() { console.log("Cat meows"); }  
}  
  
let a1 = new Dog();  
let a2 = new Cat();  
a1.speak();  
a2.speak();
```

Output:

```
Dog barks  
Cat meows
```

b. Dynamic Typing (Flexible Polymorphism)

JavaScript is dynamically typed, so the same function can work with different data types.

Example:

```
function add(a, b) { return a + b; }

console.log(add(5, 10));           // 15 (Number)
console.log(add("Hello ", "JS")); // Hello JS (String)
```

Output:

```
15
Hello JS
```

c. Runtime Decision (Dynamic Dispatch)

Example:

```
class Shape { draw() { console.log("Drawing shape"); } }
class Circle extends Shape { draw() { console.log("Drawing circle"); } }

let obj = new Circle();
obj.draw(); // Drawing circle
```

Output:

```
Drawing circle
```


5. Classes (ES6)

Blueprint for Creating Objects

- Blueprint for creating objects
- Introduced in ES6
- Cleaner syntax over prototypes

Important Notes:

- Classes are syntactic sugar — internally still uses prototype

Keywords: constructor, this, extends, super

Example:

```
class Person {  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  
  greet() {  
    return `Hello, my name is ${this.name}`;  
  }  
}  
  
const p1 = new Person("Mayur", 25);  
console.log(p1.greet());
```

Output:

```
Hello, my name is Mayur
```

6. Objects

Key-Value Pairs — Real World Entities

- Collection of key-value pairs
- Represents real-world entity

Can be created using:

- Object literal {}
- Constructor function
- Class

Example:

```
const car = {  
  brand: "Toyota",  
  start() {  
    return "Car started";  
  }  
};
```

```
console.log(car.start());
```

Output:

```
Car started
```

7. Prototype & Prototype Chain

The Core of JS Inheritance

In JavaScript, every object has a `[[Prototype]]` — another object from which it inherits properties and methods.

Prototype Chain:

- Check current object → if not found → go to prototype → then to prototype's prototype → until null

Prototype Example:

```
let animal = { eats: true };  
let dog = { barks: true };  
dog.__proto__ = animal;  
console.log(dog.eats); // true (inherited)
```

Output:

```
true
```

Prototype Chain Example:

```
let animal = { eats: true };  
let dog = Object.create(animal);  
let puppy = Object.create(dog);  
console.log(puppy.eats); // true  
// Chain: puppy → dog → animal → Object → null
```

Output:

```
true
```

__proto__ vs prototype

Feature	__proto__	prototype
Used in	Objects	Functions
Purpose	Points to prototype	Defines prototype
Access	Runtime	Design time

Prototype with Constructor:

```
function Person(name) { this.name = name; }

Person.prototype.sayHello = function () {
  console.log("Hello " + this.name);
};

let p1 = new Person("Mayur");
p1.sayHello();
// Flow: p1 → Person.prototype → Object.prototype → null
```

Output:

```
Hello Mayur
```

Prototype Chain with Classes:

```
class Animal { speak() { console.log("Animal speaks"); } }
class Dog extends Animal { bark() { console.log("Dog barks"); } }

let d = new Dog();
d.speak(); // from Animal
d.bark();  // from Dog
```

8. this Keyword

Refers to Current Execution Context

Value depends on: Object (refers to object), Function (depends on call), Arrow function (inherits from parent).

i. Inside an Object (Method)

```
let user = { name: "Mayur", greet() { console.log(this.name); } };  
user.greet();
```

Output:

```
Mayur
```

ii. Inside a Regular Function

```
function show() { console.log(this); }  
  
show();  
  
// In browser → window | In strict mode → undefined
```

Function inside object (loses context):

```
let obj = { name: "JS", show: function () {  
    function inner() { console.log(this.name); }  
    inner();  
}};  
obj.show(); // undefined
```

iii. With Constructor Function

```
function Person(name) { this.name = name; }  
  
let p1 = new Person("Mayur");  
console.log(p1.name); // Mayur
```

Output:

```
Mayur
```

iv. In Event Handlers

```
button.addEventListener("click", function () {  
    console.log(this); // button element  
});
```

v. Arrow Functions

Arrow functions do NOT have their own this — they inherit it from parent (lexical scope).

Arrow function (works — inherits this from show()):

```
let obj = { name: "Mayur", show: function () {  
    let inner = () => { console.log(this.name); };  
    inner();  
}};  
obj.show(); // Mayur
```

Output:

```
Mayur
```

Normal function (fails — loses this):

```
let obj = { name: "Mayur", show: function () {  
    function inner() { console.log(this.name); }  
    inner();  
}};  
obj.show(); // undefined
```

9. Static Methods

Belong to the Class — Not to Objects

- Static methods belong to the class itself, not to instances
- Called using class name
- Defined using the static keyword inside a class

Important Notes:

- Cannot access instance variables directly
- Used for: Utility methods & Helper functions

Basic Example:

```
class MathUtils {  
    static add(a, b) { return a + b; }  
}  
  
console.log(MathUtils.add(5, 3)); // 8  
// Called using class → MathUtils.add()  
// Not allowed → new MathUtils().add()
```

Output:

8

Static vs Instance Method

Feature	Static Method	Instance Method
Belongs to	Class	Object
Call using	Class name	Object
Access this	Class	Instance
Use case	Utility/helper	Object behavior

Use Cases of Static Methods

1) Utility Functions:

```
class Calculator { static square(num) { return num * num; } }
```

```
console.log(Calculator.square(4)); // 16
```

Output:

```
16
```

2) Common Reusable Logic:

```
class StringUtils { static toUpper(str) { return str.toUpperCase(); } }  
console.log(StringUtils.toUpper("hello")); // HELLO
```

Output:

```
HELLO
```

3) Factory Methods:

```
class User {  
  constructor(name) { this.name = name; }  
  
  static createGuest() { return new User("Guest"); }  
}  
  
let guest = User.createGuest();  
console.log(guest.name); // Guest
```

Output:

```
Guest
```

4) Validation / Business Logic:

```
class Validator {  
  static isEmail(email) { return email.includes("@"); }  
}  
  
console.log(Validator.isEmail("test@gmail.com")); // true
```

Output:

```
true
```

Calling Static Methods from Another Static Method

```
class Demo {  
  static m1() { console.log("m1 called"); }  
  
  static m2() { this.m1(); } // this → refers to class  
}
```



```
Demo.m2 () ;
```

Key Points:

- Defined using static keyword
- Called using class name
- Not accessible via object
- this refers to class
- Cannot directly access instance properties
- Used for utility/helper logic
- Can call other static methods using this

10. Constructor Function (Old Way)

Before ES6 Classes — The Blueprint Pattern

- Before ES6 classes, JavaScript used constructor functions to create objects
- A constructor function is a normal function used with the new keyword

Important Notes:

- Use new keyword
- Prototype used for shared methods
- Constructor functions start with a Capital letter

Example:

```
function Person(name, age) {  
    this.name = name;  
    this.age = age;  
}
```

```
let p1 = new Person("Mayur", 25);  
let p2 = new Person("Rahul", 30);
```

```
console.log(p1.name); // Mayur  
console.log(p2.name); // Rahul
```

Output:

```
Mayur  
Rahul
```

What Happens When You Use new?

- Creates a new empty object {}
- Sets prototype → Person.prototype
- Assigns this → new object
- Executes function code
- Returns the object

Wrong Way (Memory Waste):

```
function Person(name) {
```

```
this.name = name;
this.sayHello = function () {
    console.log("Hello " + this.name);
};
}
// Each object gets separate copy of function → wastes memory
```

Correct Way (Using Prototype):

```
function Person(name) { this.name = name; }
```

```
Person.prototype.sayHello = function () {
    console.log("Hello " + this.name);
};
```

```
let p1 = new Person("Mayur");
```

```
let p2 = new Person("Rahul");
```

```
p1.sayHello(); // Hello Mayur
```

```
p2.sayHello(); // Hello Rahul
```

Output:

```
Hello Mayur
```

```
Hello Rahul
```

11. Functional Inheritance

Factory Functions — Composition Over Inheritance

- Uses factory functions instead of classes or constructor functions
- No class, no this — uses closures
- Focuses on composition over inheritance

Basic Factory Function:

```
function createUser(name) {  
  return {  
    name: name,  
    greet() { console.log("Hello " + name); }  
  };  
}  
  
let u1 = createUser("Mayur");  
u1.greet(); // Hello Mayur
```

Output:

```
Hello Mayur
```

Functional Inheritance via Composition

```
function Animal(name) {  
  return { speak() { console.log(name + " makes sound"); } };  
}  
  
function Dog(name) {  
  let animal = Animal(name);  
  return { ...animal, bark() { console.log(name + " barks"); } };  
}  
  
let d = Dog("Rocky");  
d.speak(); // Rocky makes sound  
d.bark();  // Rocky barks
```

Output:

Rocky makes sound
Rocky barks

Closures for Private Data

function createBankAccount(balance) { return { deposit(amount) { balance += amount; console.log("Balance:", balance); }, getBalance() { return balance; } }; }
let acc = createBankAccount(1000); acc.deposit(500); // Balance: 1500 console.log(acc.getBalance()); // 1500 console.log(acc.balance); // undefined (private)

Output:

Balance: 1500
1500
undefined

Flexible Composition

function canEat() { return { eat() { console.log("Eating..."); } }; }
function canWalk() { return { walk() { console.log("Walking..."); } }; }
function Person() { return { ...canEat(), ...canWalk() }; }
let p = Person(); p.eat(); p.walk();

Output:

Eating...

Walking...

Functional vs Prototpal vs Classical

Feature	Functional	Prototpal	Classical
Uses class	✗	✗	✓
Uses this	✗	✓	✓
Private data	✓	✗	✗
Memory efficient	✗	✓	✓
Flexibility	High	High	Medium

12. Method Overriding

Child Replaces Parent Method

- A child class provides its own implementation of a method that already exists in the parent class
- Parent and child both have same method name
- The method that runs depends on the object at runtime

Basic Example:

```
class Animal { speak() { console.log("Animal makes a sound"); } }  
class Dog extends Animal { speak() { console.log("Dog barks"); } }
```

```
let d = new Dog();  
d.speak(); // Dog barks
```

Output:

```
Dog barks
```

Using super (Access Parent Method)

```
class Animal { speak() { console.log("Animal makes a sound"); } }
```

```
class Dog extends Animal {  
  speak() {  
    super.speak(); // call parent method  
    console.log("Dog barks");  
  }  
}
```

```
let d = new Dog();  
d.speak();
```

Output:

```
Animal makes a sound  
Dog barks
```

Constructor Overriding

```
class Animal { constructor(name) { this.name = name; } }
```

```
class Dog extends Animal {  
    constructor(name) { super(name); } // must call parent constructor  
}
```

```
let d = new Dog("Tommy");  
console.log(d.name); // Tommy
```

Output:

```
Tommy
```

Example with Prototype (Old Way)

```
function Animal() {}  
Animal.prototype.speak = function () { console.log("Animal speaks"); };
```

```
function Dog() {}  
Dog.prototype = Object.create(Animal.prototype);  
Dog.prototype.speak = function () { console.log("Dog barks"); };
```

```
let d = new Dog();  
d.speak(); // Dog barks
```

Output:

```
Dog barks
```


13. Method Overloading □

Not Natively Supported in JavaScript

In languages like Java/C++, method overloading = same function name, different number/type of parameters. JavaScript does NOT support this natively.

Why JS Does NOT Support True Overloading:

- Functions are objects
- If you define multiple functions with same name → last one overrides

```
function add(a) { return a; }  
function add(a, b) { return a + b; }  
console.log(add(5)); // NaN (second function used)
```

i. Using arguments Object

```
function add() {  
    if (arguments.length === 1) return arguments[0];  
    else if (arguments.length === 2) return arguments[0] + arguments[1];  
}  
console.log(add(5)); // 5  
console.log(add(5, 10)); // 15
```

Output:

```
5  
15
```

ii. Using Default Parameters

```
function add(a, b = 0) { return a + b; }  
console.log(add(5)); // 5  
console.log(add(5, 10)); // 15
```

Output:

```
5  
15
```

iii. Using Rest Parameters (Advanced)

```
function add(...nums) { return nums.reduce((sum, n) => sum + n, 0); }  
console.log(add(5));           // 5  
console.log(add(5, 10, 15)); // 30
```

Output:

5

30

Key Points:

- JavaScript does NOT support true overloading
- Last function overrides previous ones with same name
- Use conditions, undefined checks, arguments or ...rest to simulate

14. Object Creation Methods

5 Ways to Create Objects in JavaScript

i. Object Literal {} (Most Common)

```
let user = { name: "Mayur", age: 25 };  
console.log(user.name); // Mayur
```

Output:

```
Mayur
```

ii. new Object()

```
let user = new Object();  
user.name = "Mayur";  
user.age = 25;  
console.log(user.name);
```

Output:

```
Mayur
```

iii. Constructor Function (Old Way)

```
function Person(name, age) { this.name = name; this.age = age; }  
let p1 = new Person("Mayur", 25);
```

iv. ES6 Class (Modern Way)

```
class Person { constructor(name, age) { this.name = name; this.age = age; } }  
let p1 = new Person("Mayur", 25);
```

v. Object.create() — Very Important

Creates a new object using an existing object as prototype.

```
let animal = { eats: true };  
let dog = Object.create(animal);
```

```
dog.barks = true;
console.log(dog.eats); // true (inherited)
// dog → animal → Object.prototype → null
```

Output:

```
true
```

Adding Properties with Object.create():

```
let person = Object.create({}, {
  name: { value: "Mayur" },
  age: { value: 25 }
});
console.log(person.name); // Mayur
```

Prototypal Inheritance Example:

```
let parent = { greet() { console.log("Hello"); } };
let child = Object.create(parent);
child.greet(); // Hello
```

{ } vs Object.create() — Comparison

Feature	{ }	Object.create()
Prototype	Object.prototype	Custom
Flexibility	Low	High
Control	Limited	Full control

Key Points:

- { } → simplest way
- new Object() → rarely used
- Constructor → reusable (old way)
- Class → modern approach
- Object.create() → prototype control, used for inheritance

Quick Reference

One-Line Memory Tricks

Concept	Memory Trick	Key Keyword(s)
Encapsulation	Hide data	#private, get(), set()
Abstraction	Hide logic	class, function
Inheritance	Reuse code	extends, super
Polymorphism	Same method, different behavior	override, extends
Prototype	Core of JS inheritance	__proto__, prototype
this	Current execution context	this, bind, call, apply
Static Methods	Class-level utility	static
Constructor Function	Old-way blueprints	new, this, prototype
Functional Inheritance	Compose, don't extend	factory, spread (...)
Method Overriding	Child replaces parent	extends, super
Method Overloading	Simulated in JS	arguments, ...rest, defaults
Object.create()	Prototype control	Object.create()